



Kafka

Prof. Thiago Nogueira

Nov/2023

Agenda

Batch vs
Streaming

Arquitetura

All
Together

Kafka

Cenários

Atividades



01

Batch vs
Streaming



Batch vs Streaming

- Batch:
 - Dataset fixo
 - Depois que o evento ocorre (at rest)
 - Estatísticas de um grupo numa sala fechada
- Streaming:
 - Dados de entrada contínua
 - Enquanto o evento ocorre (in motion)
 - Estatísticas de um grupo numa maratona

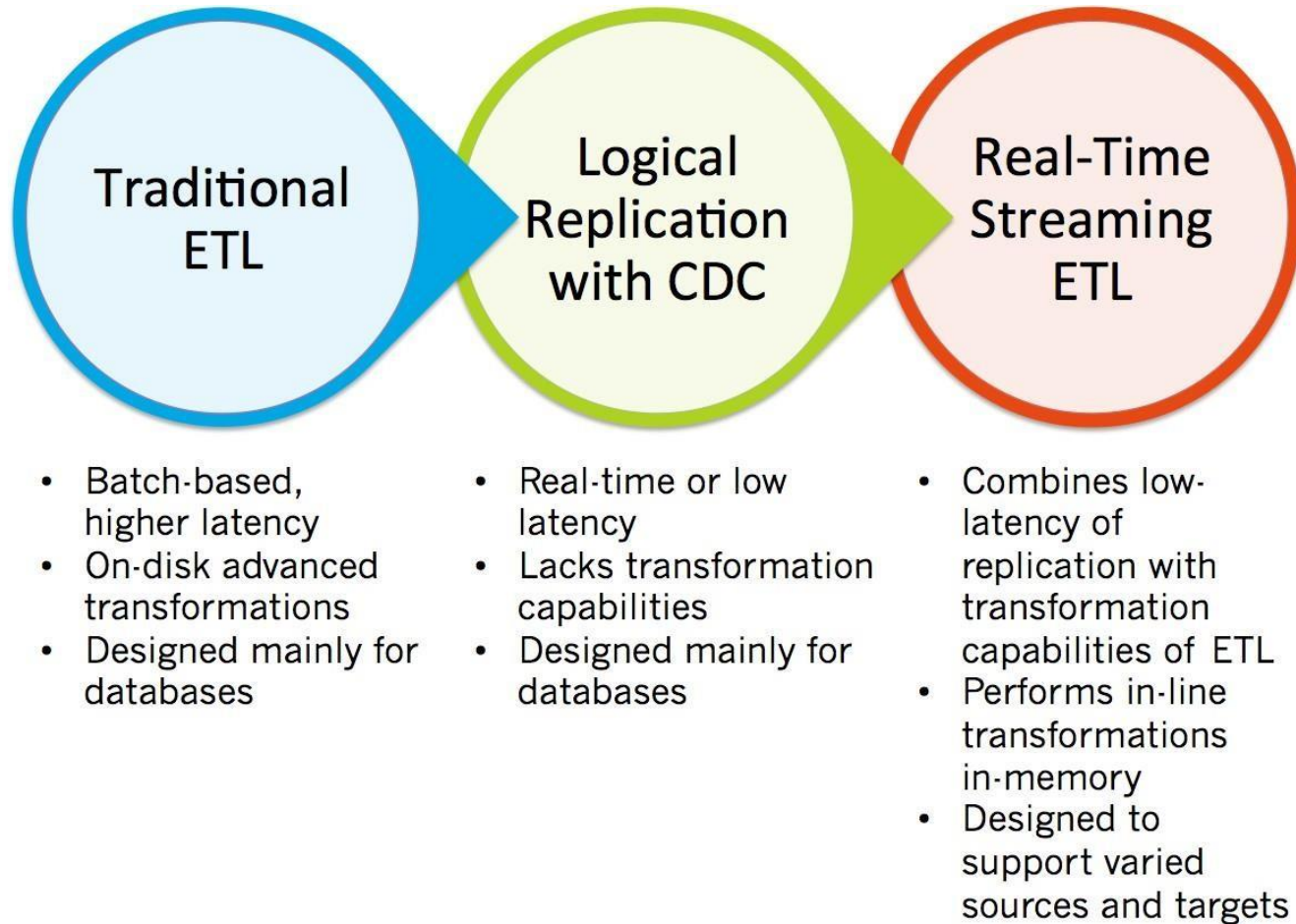
Processamento Batch

- Características:
 - Acesso aos dados completos
 - Devolve a resposta **ao final do processo**
 - Permite fazer análises complexas (ex: treinamento de modelos)
 - Otimizado para throughput (dados processados por segundo)
- Exemplos:
 - Sumário de vendas do ultimo mês
 - Treinamento de modelos para detecção e spam
 - Frameworks: ETL tradicional, Map Reduce, Spark

Processamento Streaming

- Características:
 - Acesso ao fluxo de dados
 - Devolve a resposta **nesse momento**
 - Processa dados de 1 registro ou pequenas janelas
 - Computações relativamente simples
 - Otimizado para latência (tempo médio para um registro)
- Exemplos:
 - Detecção de fraudes
 - Utilização de um modelo pré-treinado para rotular uma reclamação
 - Frameworks: Apex, Storm, Spark Streaming

Replicação de Dados



Tipos de Captura

- **Batch:** Operação de processo de carga agendada para ocorrer automaticamente em um horário definido ou em intervalos de tempos regulares
- **Tempo real:** As fontes são monitoradas constantemente e as operações são capturadas no momento em que ocorrem
 - Exigências de negócio tem aumentado a demanda por este tipo de captura
 - **CDC (Change Data Capture)** – Conjunto de design patterns para determinar e rastrear os dados que mudaram para que a ação possa ser tomada a partir da alteração

Tipos de captura

Table 1. Classification of Loading Processes (cf. Farooq and Sarwar 2010 and Kotopoulos 2012)

ETL-Types	Traditional ETL	Near Real-Time ETL		ETL-DDF	ETL-RDC
Mode	Batch	Incremental / Mini-Batch	Incremental / Micro-Batch	Incremental / Real-Time	Incremental / Real-Time
Description	Data is loaded in full or incrementally using a off-peak window	Data is loaded incrementally using intra-day loads	Source changes are captured and accumulated to be loaded in intervals	Source changes are captured and immediately applied to the DW	Source changes are captured and immediately applied to the DW
frequency	monthly, weekly, daily	daily, twice a day, hourly, minutes	15 minutes or higher	minutes, seconds	minutes, seconds
data volumes	high	low	low	low	low
Latency	hours	minutes	minutes	minutes, seconds	minutes, seconds
Capture	filter query	filter query	CDC	CDC	CDC
Initialization	Pull	Pull	Push, then Pull	Push	Push
Target Load	High impact	Low Impact, load frequency is tuneable			
Source Load	High impact	Queries at peak times necessary	Some to none depending on CDC technique		

O Caso da Limonada

Livia abre uma venda de limonada em frente à sua casa. **Tânia**, sua irmã, possui a tarefa de informar à sua mãe sobre o progresso de vendas e assegurar que tenham insumos suficientes para garantir a venda de limonada a qualquer tempo

O Caso da Limonada

Batch:

Tânia consulta (QUERY) **Livia** como foram as vendas ao final de cada dia. **Livia** possui uma série de atividades para contabilizar o resultado final do dia. Com este resultado, elas podem estimar o faturamento do próximo dia e a quantidade de limonada que deve ser feita para o dia seguinte

O Caso da Limonada

Mini-Batch:

Percebendo que precisa de melhor visibilidade das operações ao longo do dia, Tânia nota que seria melhor ter esse resultado a cada 15 minutos, mas um limite prático torna-se aparente:

- Quando **Livia** mal terminou de consolidar os dados, já precisa iniciar uma nova consolidação
- O método de coleta torna o processo ineficiente, pois o trabalho adicional durante a jornada atrapalha no atendimento aos consumidores

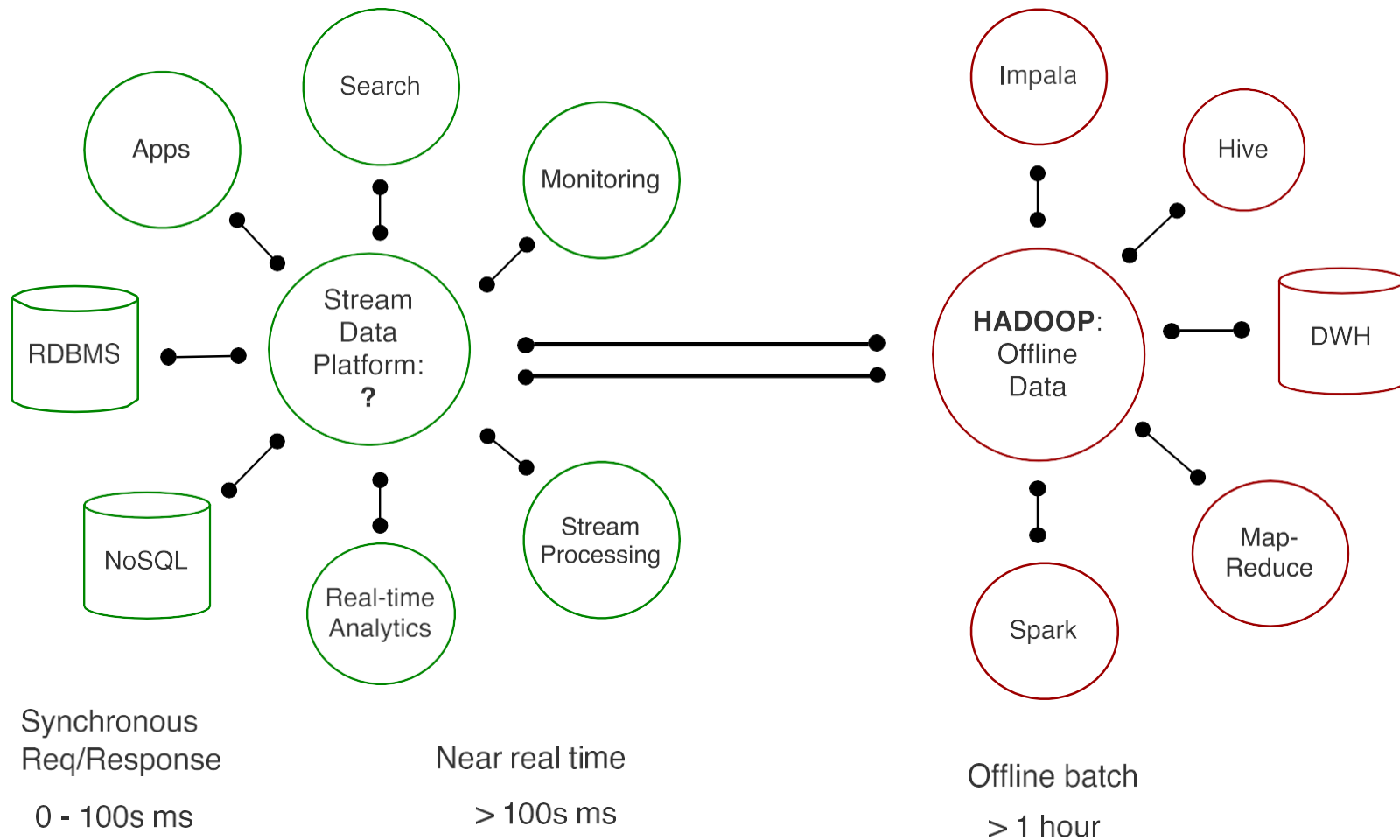
O Caso da Limonada

Tempo-Real:

Tânia passa a trabalhar ao lado de sua irmã e coleta cada nota fiscal. Imediatamente entrega à sua mãe.

- Neste caso, a transição do formato Batch para CDC, não só tornou mais eficiente o trabalho de **Lívia**, que pode se dedicar ao atendimento aos clientes, como deu à sua mãe uma visão instantânea sobre o progresso de vendas

Stream Data Platform



02

Arquitetura



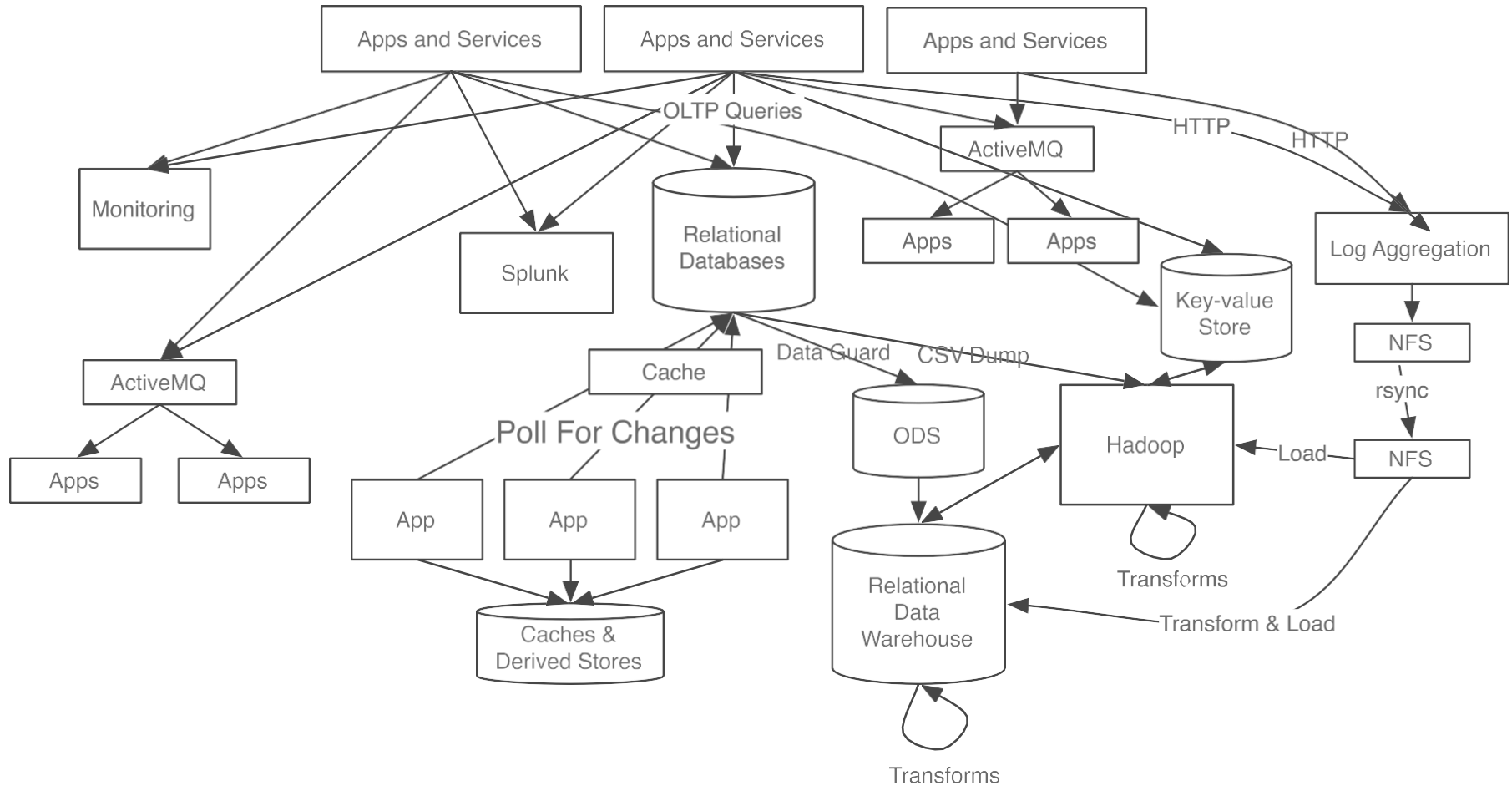
This is a mess



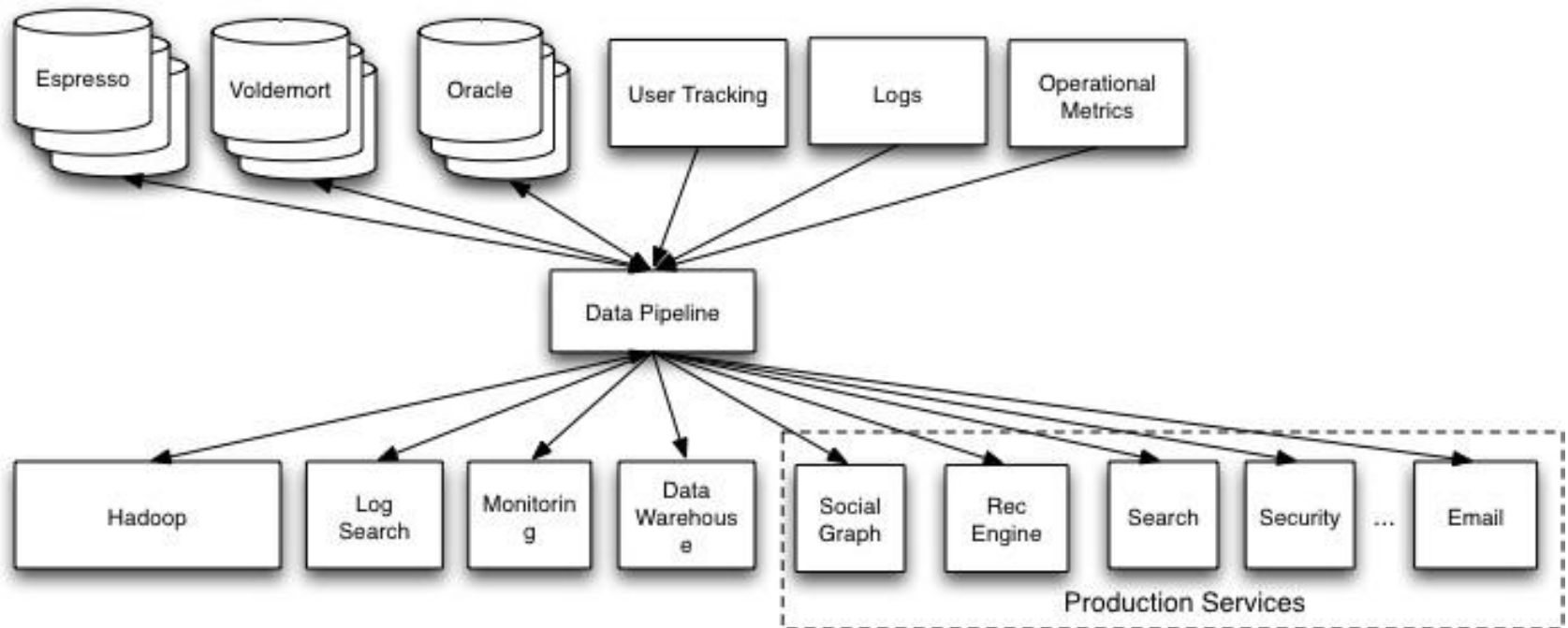
This is a big mess



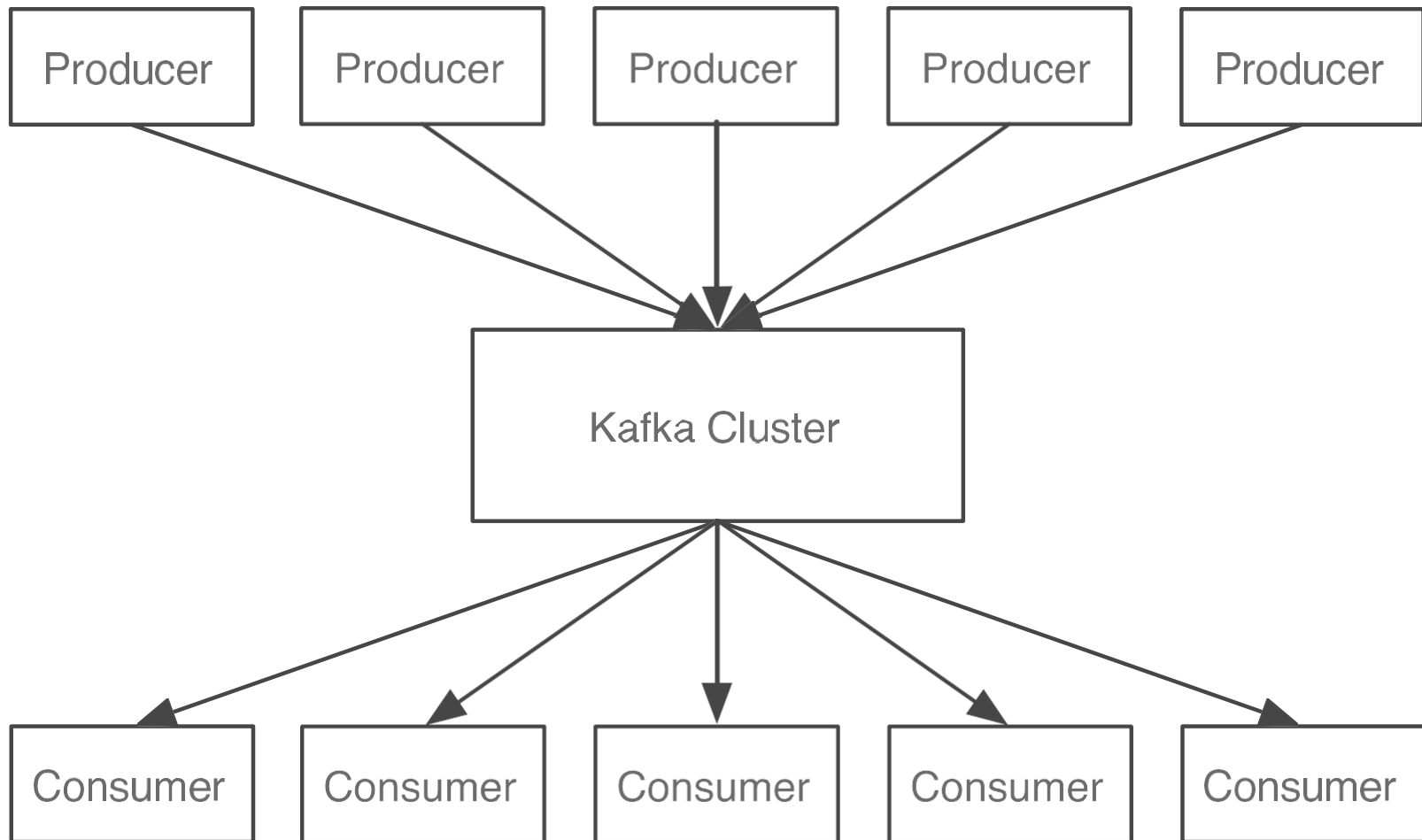
This is a giant mess



What we'd really like



How we got it

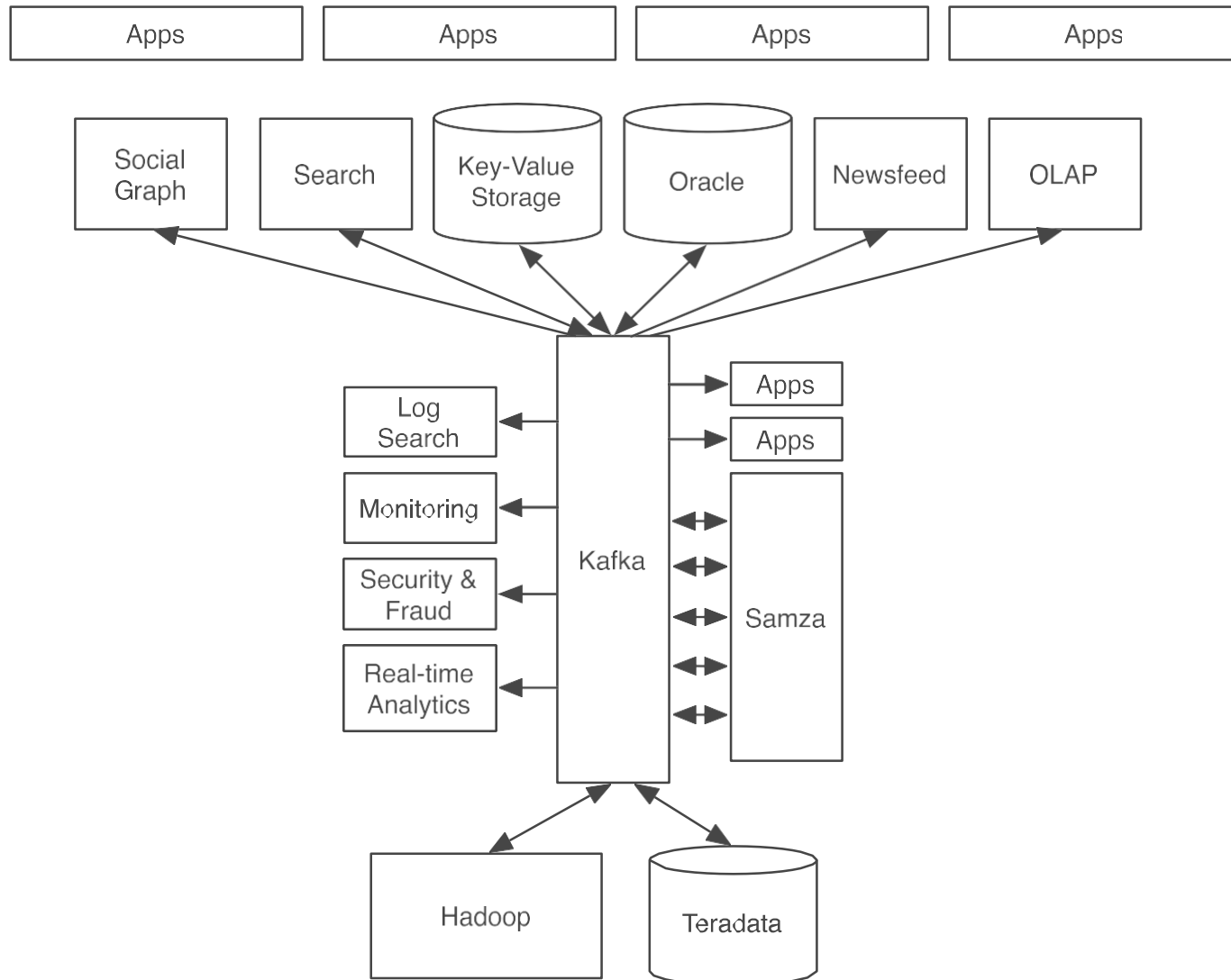




03

All Together

Kafka Stream Data Platform



LinkedIn

- Everything in the company is a real-time stream
- > 500 billion messages written per day
- > 2.5 trillion messages read per day
- ~ 1 PB of stream data
- Tens of thousands of producer processes
- Backbone for data stores
 - Search
 - Social Graph
 - Newsfeed
- Basis for stream processing

Elsewhere

- Netflix: real-time monitoring and event processing
- Twitter: as part of their Storm real-time data pipelines
- Spotify: log delivery (from 4h down to 10s), Hadoop
- Loggly: log collection and processing
- Mozilla: telemetry data
- Airbnb, Cisco, Gnip, InfoChimps, Ooyala, Square, Uber, ...

Elsewhere

intuit.

airbnb

ebay

Square

Cerner

yelp



UBER



WIKIPEDIA
The Free Encyclopedia

Y!

PayPal

CISCO

dish
NETWORK

Pinterest

Adobe

verizon

Goldman
Sachs

NETFLIX

box

salesforce

Why this is the future

1. System diversity is increasing
2. Data diversity and volume is increasing
3. The world is getting faster
4. The technology exists

04

Kafka



O que é Kafka

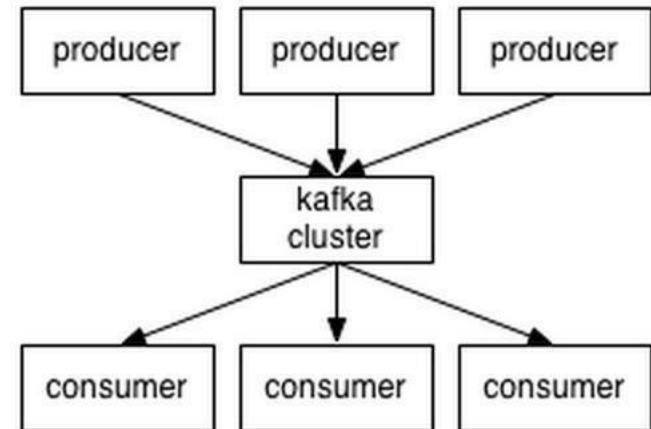
- Kafka é uma plataforma distribuída de streaming
 - Altamente escalável (partições)
 - Tolerante a falhas (réplicas)
 - Permite alto nível de paralelismo e desacoplamento entre producers e consumers
- Desenvolvimento iniciado pelo LinkedIn em 2009 e incubado na ASF desde 2012
- Em 2014 o LinkedIn criou uma nova empresa chamada Confluent para alavancar o projeto
- Desenvolvido em Scala

Características

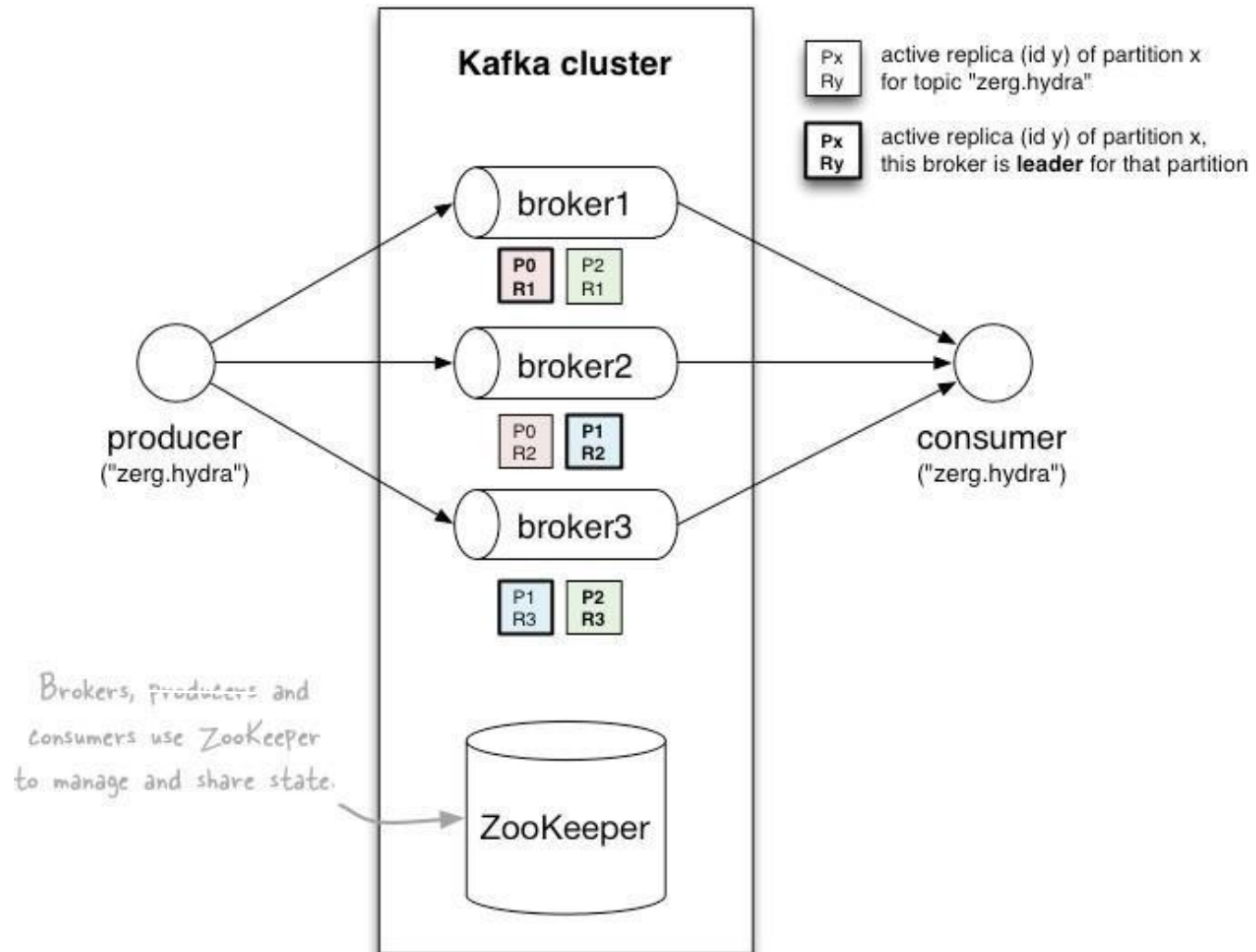
- Alto throughput, trabalhando com HW commodity e suportando milhões de mensagens por segundo. Sua rapidez é baseada na escrita em page cache do SO e persistência dos dados em disco. Aleitura também é realizada do page cache.
- Camada de persistência para as mensagens, podendo manter a mensagem armazenada por um tempo determinado.
- Possui APIs com suporte a clientes de diferentes plataformas: Java, .Net, PHP, Ruby e Python.
- Processamento em tempo real, mensagens produzidas são imediatamente visíveis pelos Consumidores
- Suporta muitos consumidores em paralelo.
- Possui recuperação automática se algum broker falhar.
- Utiliza o modelo Push/Pull.
- Não é uma solução end-user. Na maioria dos casos, é necessário escrever o código para os producers e consumers.
- Não é voltado a transformação ou criptografia de dados.

A first look

- The who is who
 - **Producers** write data to **brokers**.
 - **Consumers** read data from **brokers**.
 - All this is distributed.
- The data
 - Data is stored in **topics**.
 - **Topics** are split into **partitions**, which are **replicated**.



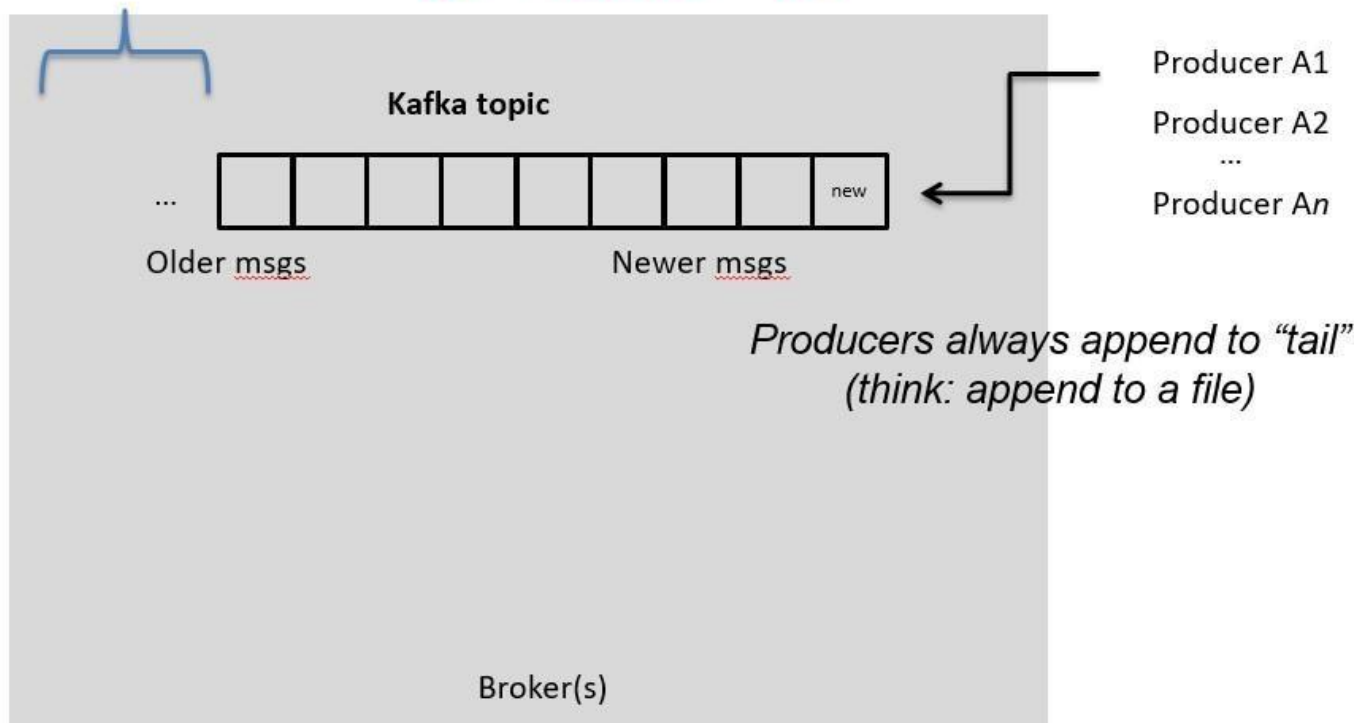
A first look



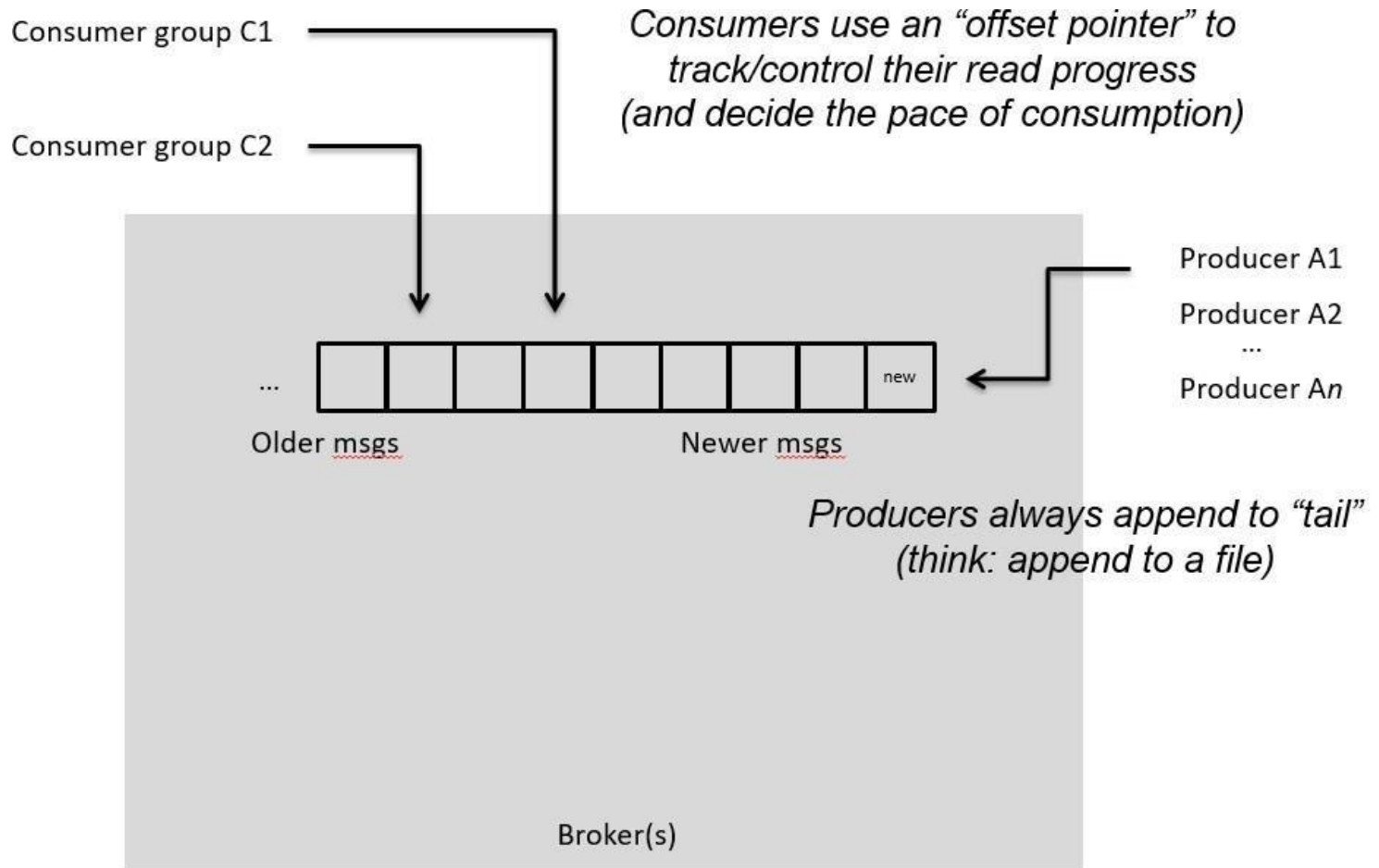
Topics

- **Topic:** feed name to which messages are published
 - Example: “zerg.hydra”

*Kafka prunes “head” based on **age** or **max size** or “**key**”*



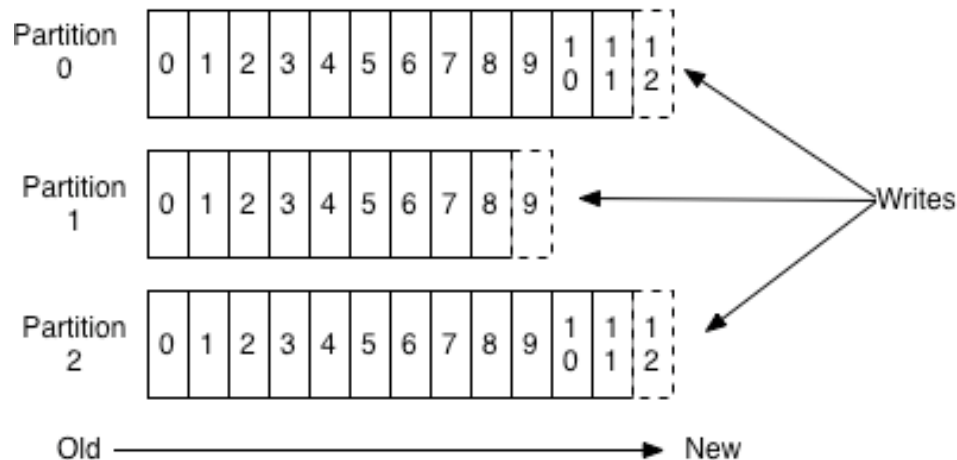
Topics



Partitions

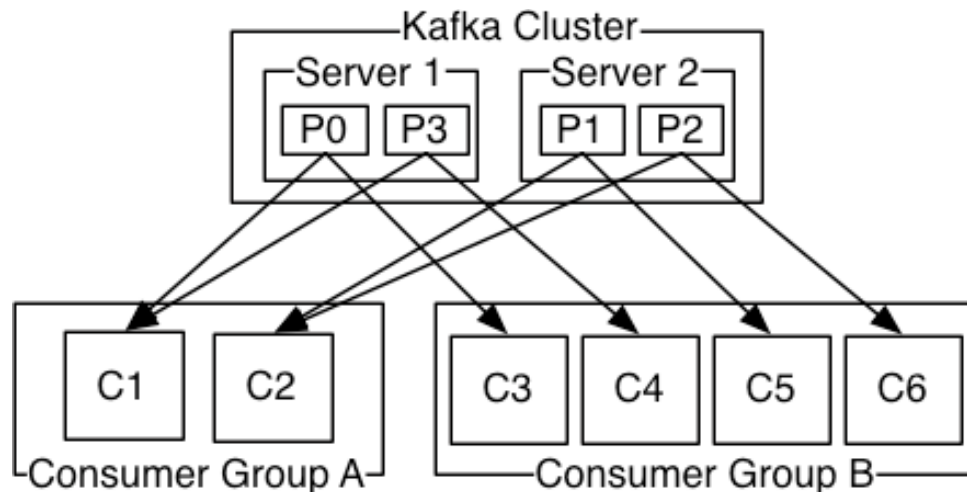
- A topic consists of **partitions**.
- Partition: **ordered + immutable** sequence of messages that is continually appended to

Anatomy of a Topic



Partitions

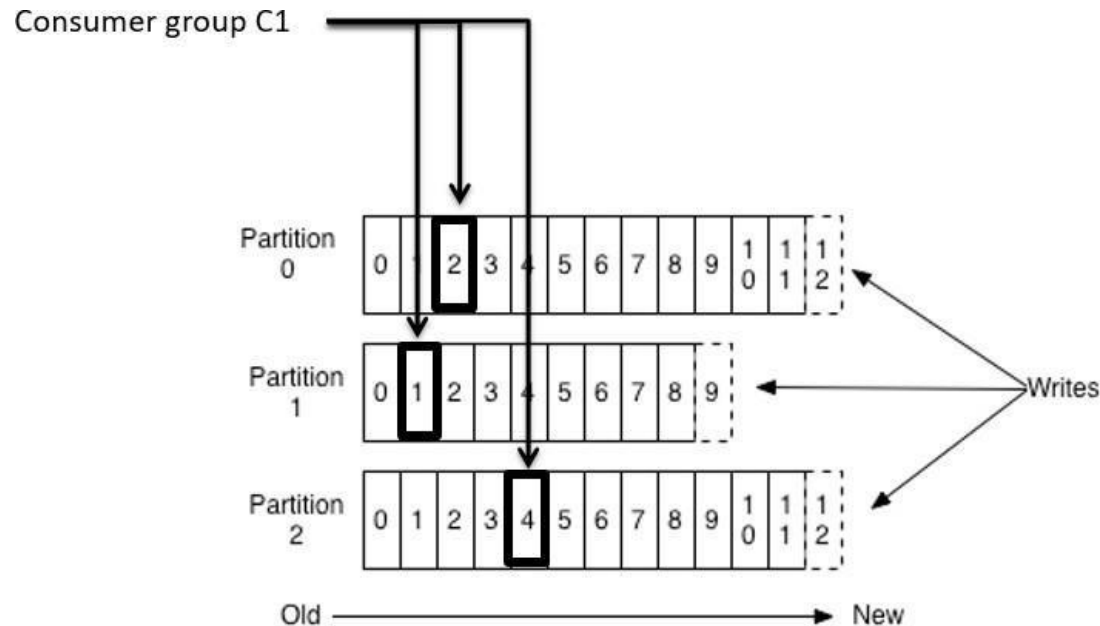
- #partitions of a topic is configurable
- #partitions determines **max** consumer (group) parallelism
 - cf. parallelism of Storm's KafkaSpout via `builder.setSpout(, , N)`



- Consumer group A, with 2 consumers, reads from a 4-partition topic
- Consumer group B, with 4 consumers, reads from the same topic

Partition offsets

- **Offset:** messages in the partitions are each assigned a unique (per partition) and sequential id called the *offset*
 - Consumers track their pointers via (*offset*, *partition*, *topic*) tuples

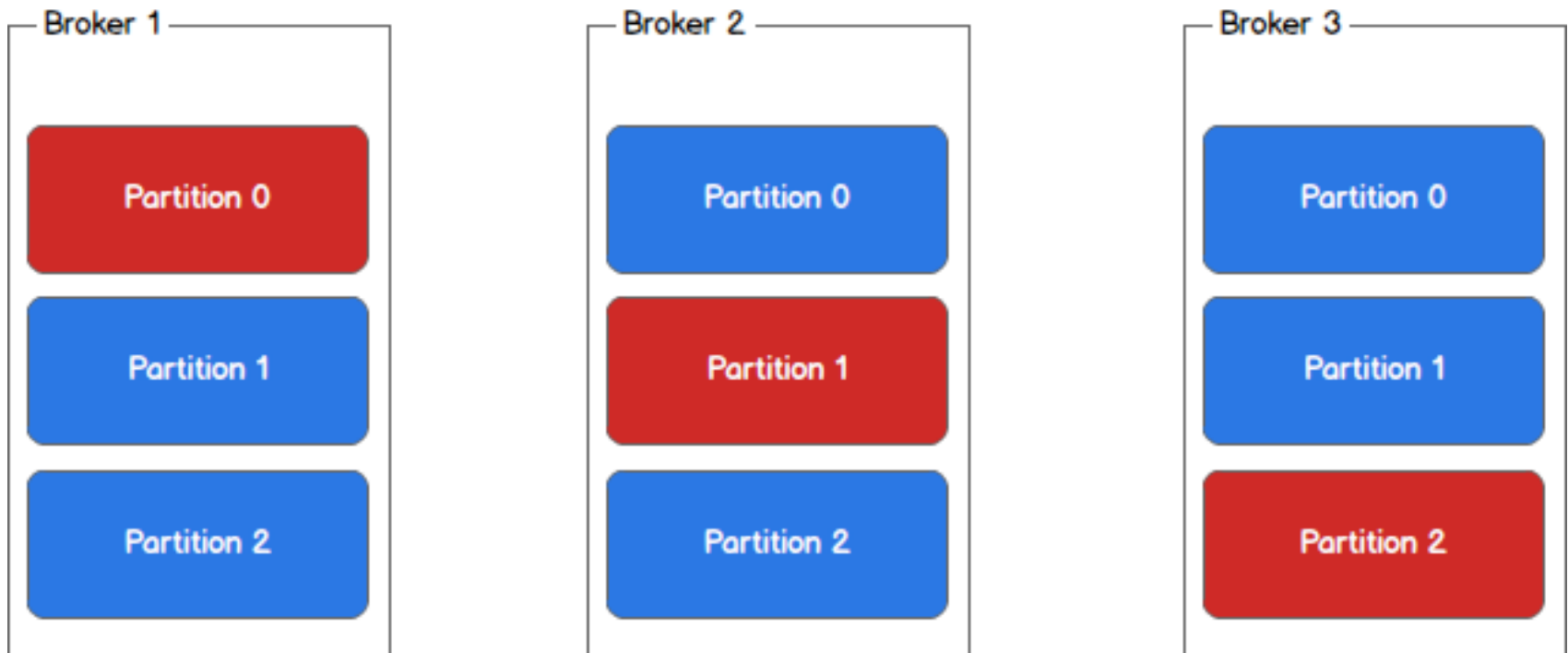


Replicas of a partition

- **Replicas:** “backups” of a partition
 - They exist solely to prevent data loss.
 - Replicas are never read from, never written to.
 - They do NOT help to increase producer or consumer parallelism!
 - Kafka tolerates (*numReplicas* - 1) dead brokers before losing data
 - LinkedIn: *numReplicas* == 2 → 1 broker can die
 - Each partition has one leader which is responsible for replicating data to the other replicas
 - Acknowledgement (ACK) levels: The number of acks the producer requires the leader to have received before considering a request complete
 - *acks=0* If set to zero then the producer will not wait for any acknowledgment from the server at all
 - *acks=1* This will mean the leader will write the record to its local log but will respond without awaiting full acknowledgement from all followers.
 - *acks=all* This means the leader will wait for the full set of in-sync replicas to acknowledge the record

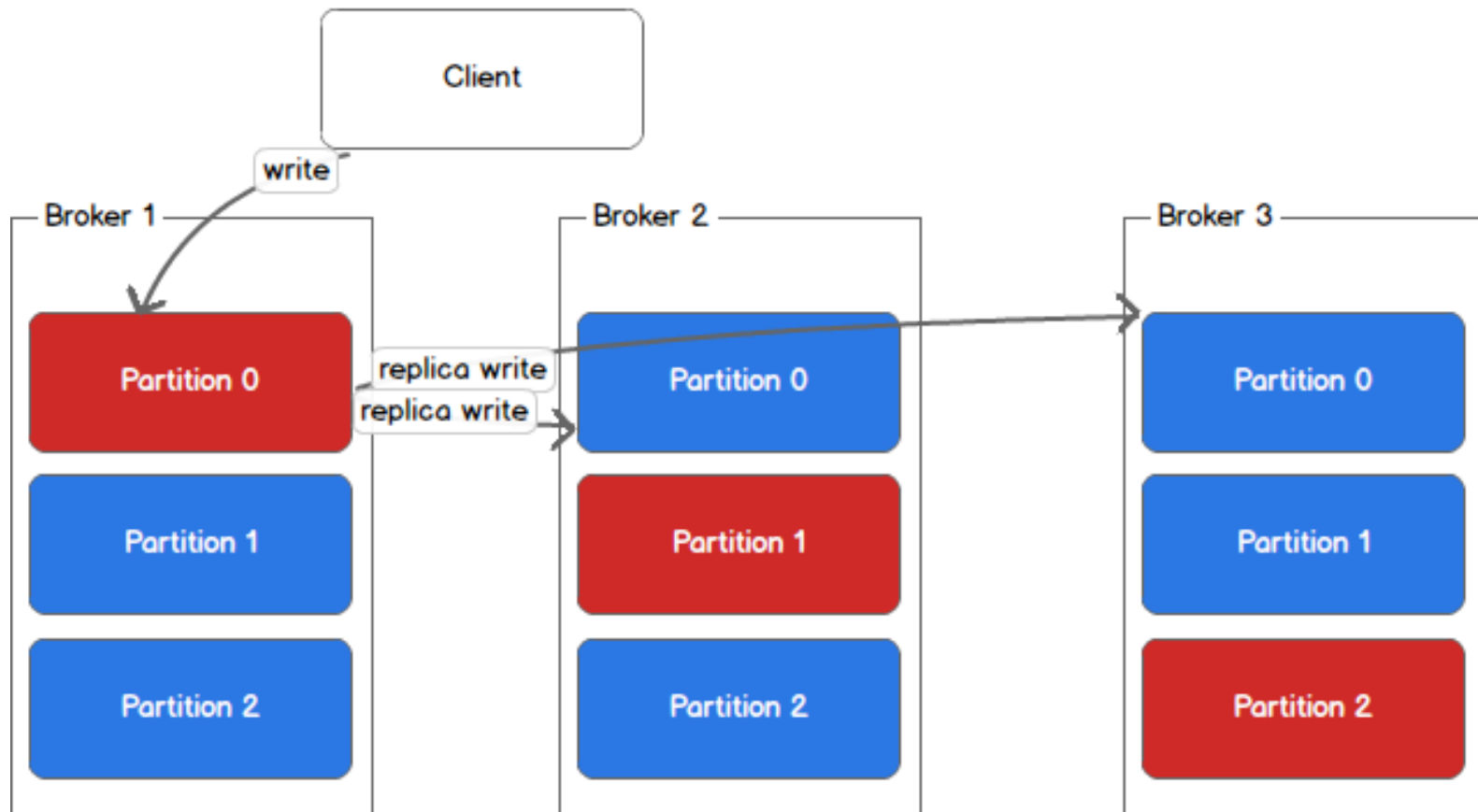
Replication Flow

Leader (red) and replicas (blue)



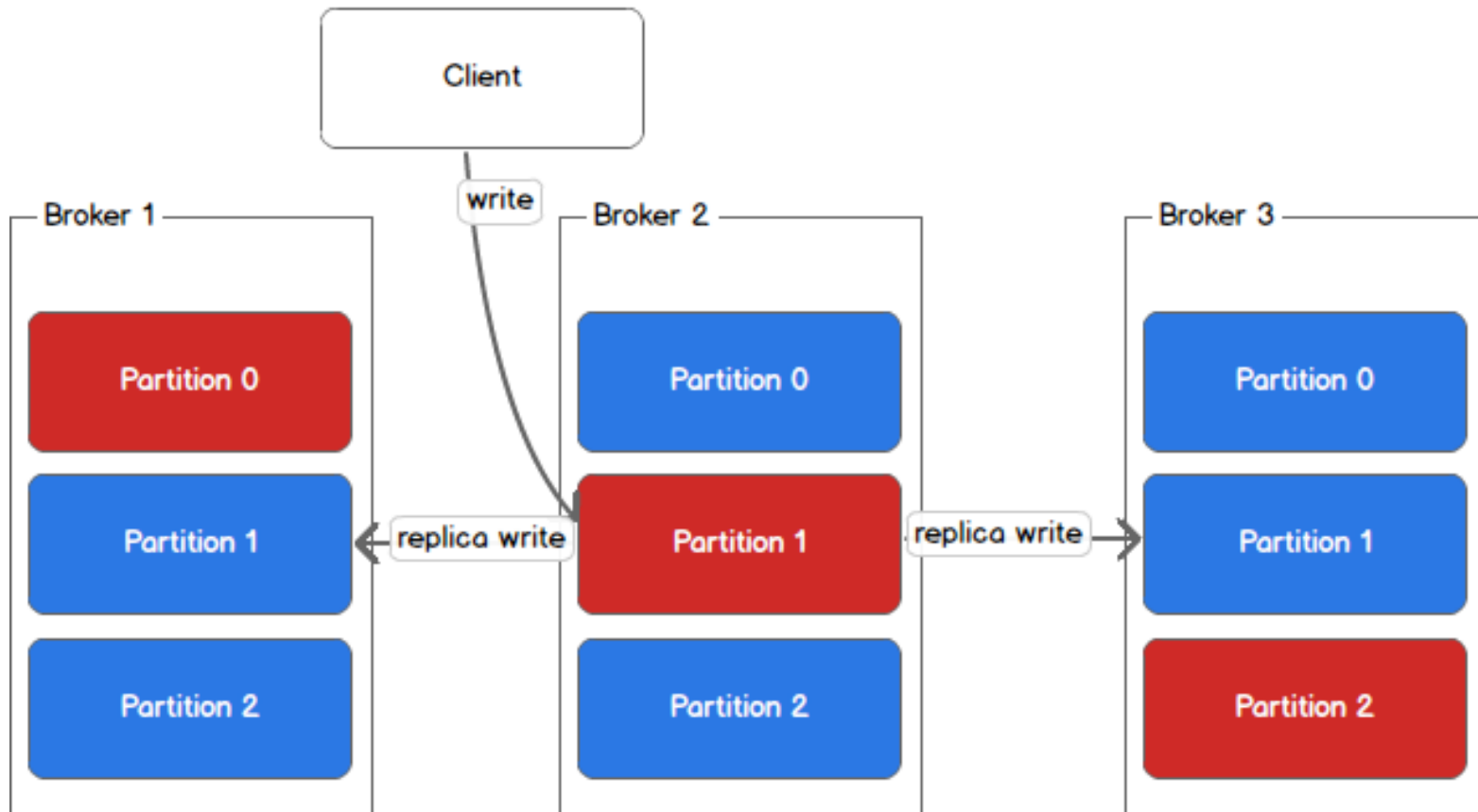
Replication Flow

Leader (red) and replicas (blue)

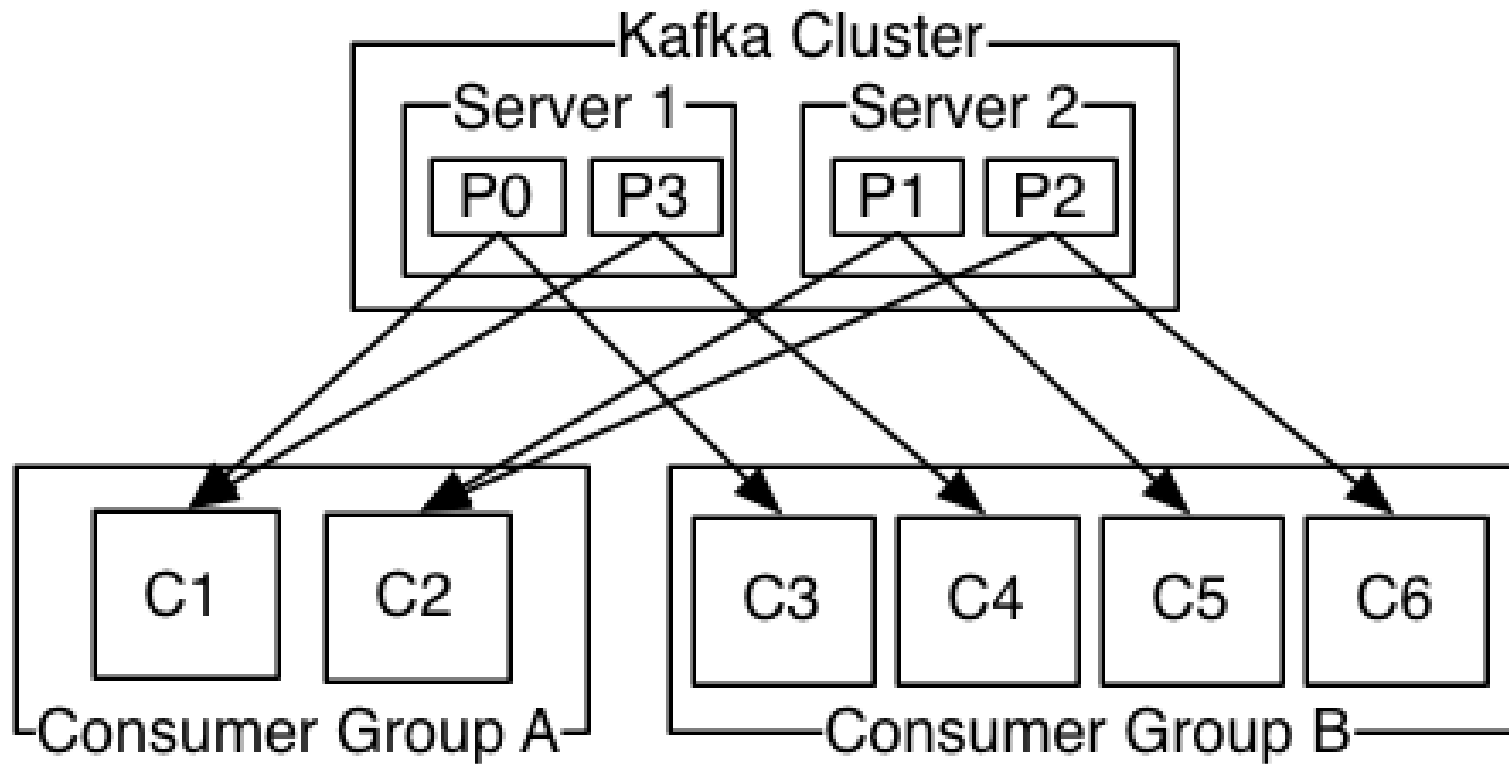


Replication Flow

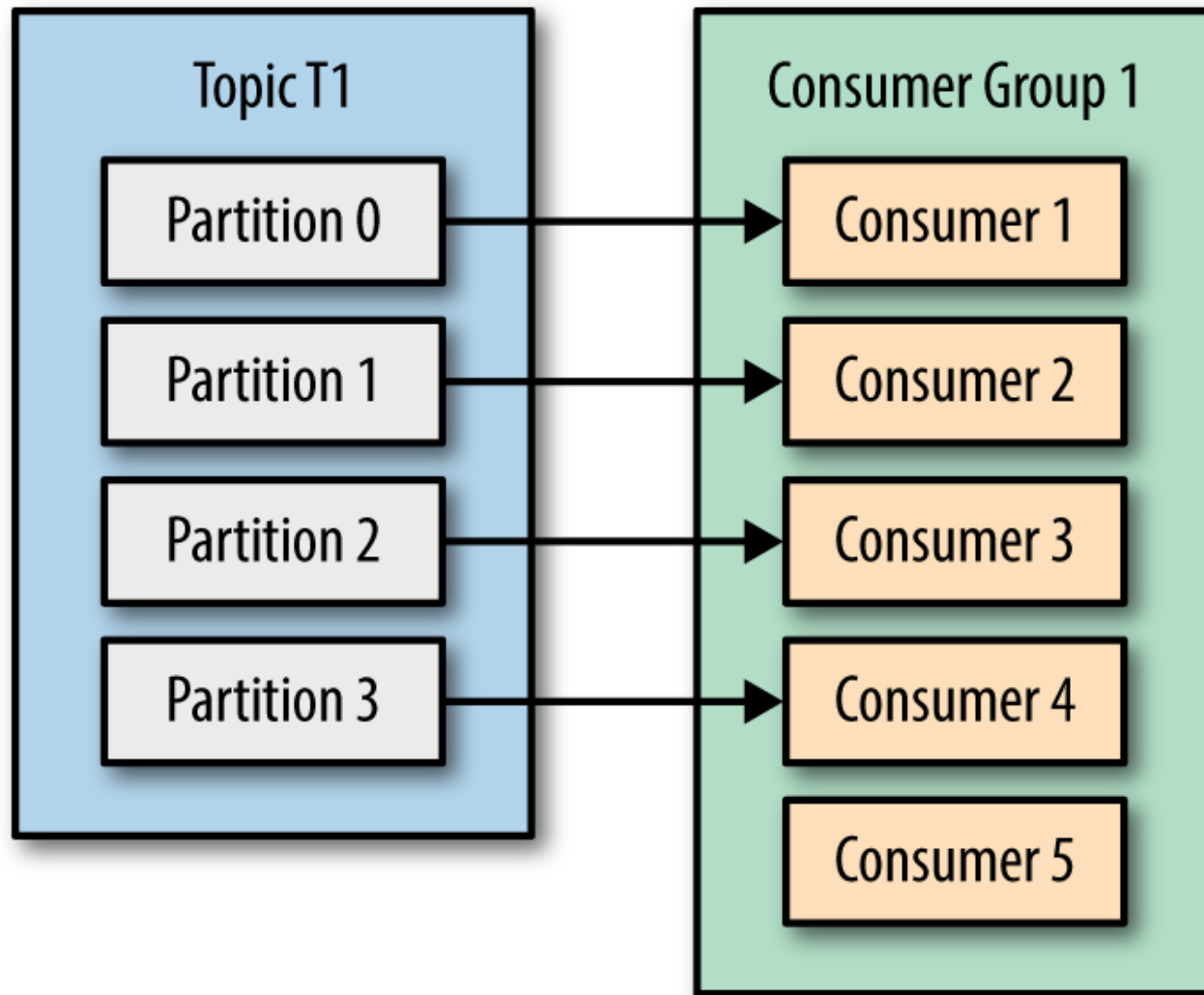
Leader (red) and replicas (blue)



Consumer Groups



Consumer Groups



05

Cenários



Oferta em tempo real

Necessidade:

- Criar ofertas para seus clientes a partir de características da navegação no site da empresa;

Solução:

- Coleta dos logs de navegação e entrega em tópico Kafka para envio de ofertas específicas.

Resultado:

- Aumento de 80% na conversão de vendas de produtos premium;

Persistência de dados

Necessidade:

- Criar rotina de contingência para solução com persistência de dados em cloud;

Solução:

- Adição de um componente Kafka na arquitetura da solução.

Resultado:

- Solução implementada de forma simples e segura, demandando o mínimo de desenvolvimento;

Escalabilidade horizontal

Necessidade:

- Ampliar a capacidade de uma plataforma de OCR de documentos e notas fiscais;

Solução:

- Utilização de um tópico Kafka particionado para que diversos consumers possam processar as imagens paralelamente.

Resultado:

- Processo paralelizado de forma simples, segura e de baixo custo, permitindo crescimento vegetativo “ilimitado”;

Muchas gracias



Consulting, Transformation, Technology and Operations